

Lecture 13: Dask, parallelism loose ends, and I/O

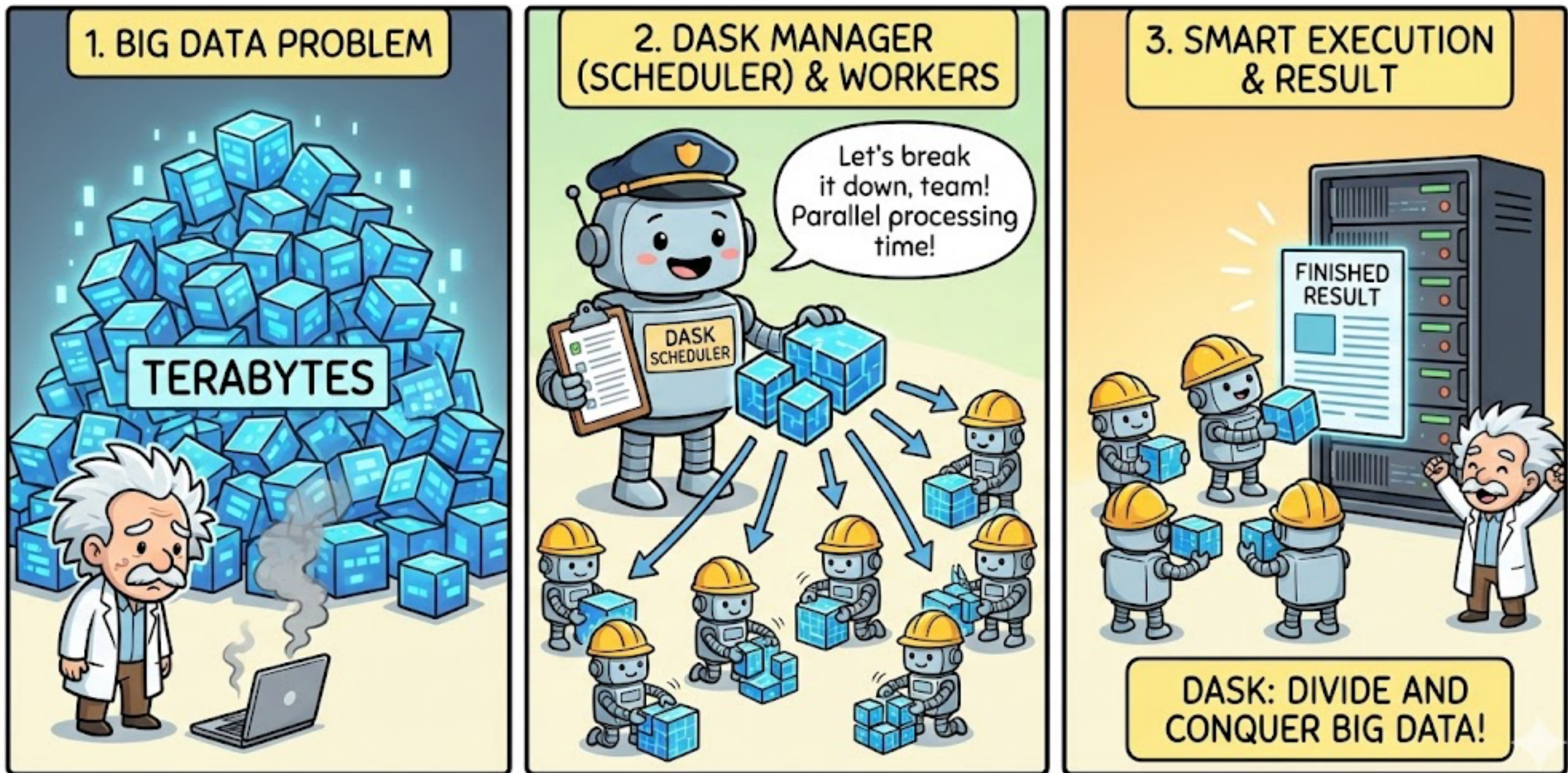
CEE 690



Dask is a library for parallel computing in Python that seamlessly breaks “Big Data” into small chunks

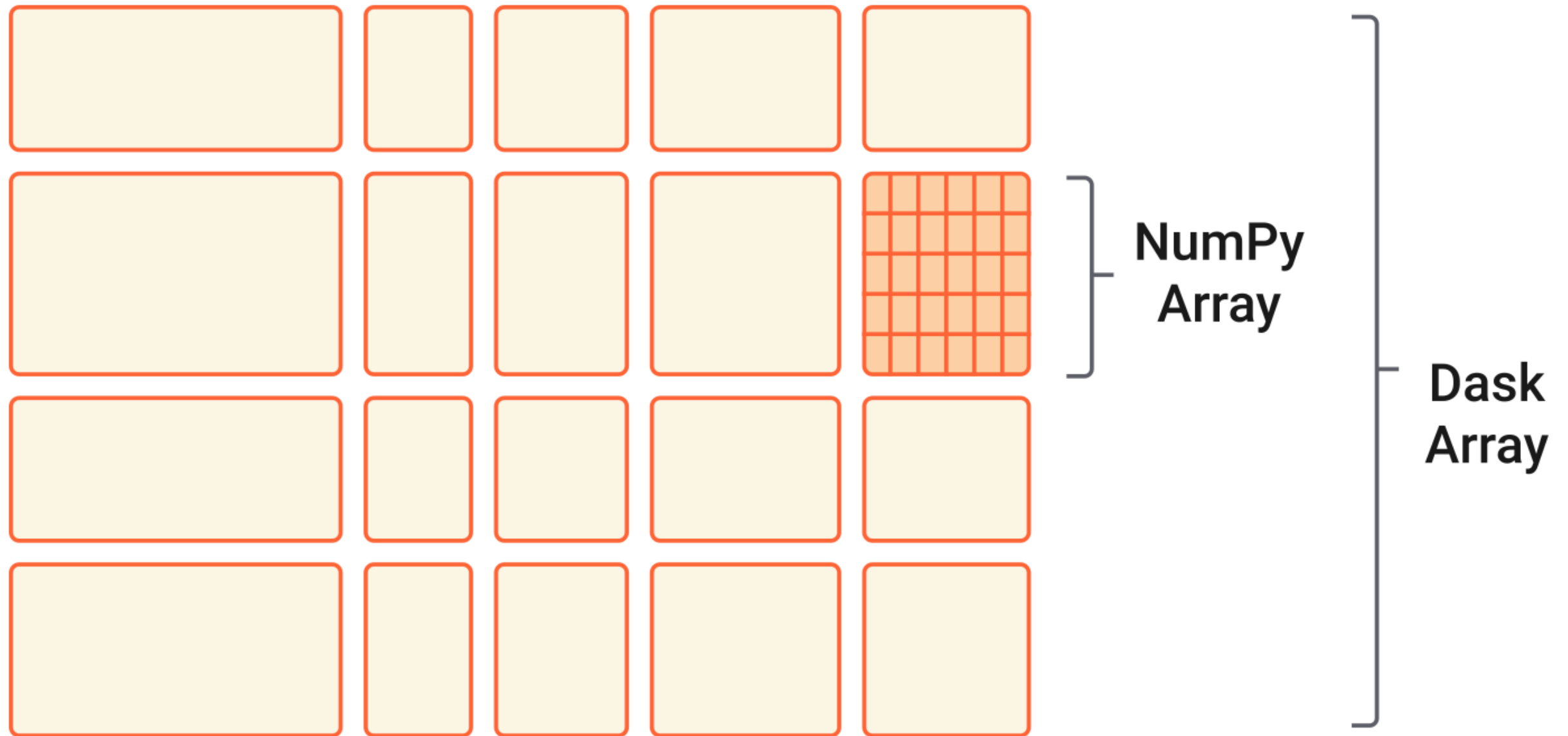
Parallelization made easy

THE DASK PROCESS



Example - If you try to read a 500 GB .tif file to one core, it will run out of memory

Dask arrays



Dask seamlessly scales

1. **Local** - Use the multiple cores/threads on compute node/laptop
2. **Distributed (single node)** - Split the compute node into multiple processes
3. **Distributed (multiple nodes)** - Run on many cores via `mpirun`

Dask local (threads)

```
# local_dask.py
from dask.distributed import Client
import dask.array as da

if __name__ == "__main__":
    # 1. Start the Local Cluster
    # This automatically uses all available cores on this specific machine
    client = Client()
    print(f"Dashboard available at: {client.dashboard_link}")

    # 2. The Science Logic
    print("Building the task graph...")
    # Simulating a 120GB 3D grid, chunked into manageable blocks
    data = da.random.random((2000, 4000, 4000), chunks=(100, 1000, 1000))

    # 3. Execution
    print("Computing the mean...")
    result = data.mean().compute()

    print(f"The mean is: {result}")

    # Clean up
    client.close()
```

```
(cee690) nc153 $ python dask_local.py
Dashboard available at: http://127.0.0.1:8787/status
Building the task graph...
Computing the mean...
The mean is: 0.4999931200013032
```

Dask distributed (MPI)

```
# 1. Initialize the MPI Network
# This acts as the "traffic cop".
# Rank 0 becomes the Scheduler, Rank 1 becomes the Worker (or Dashboard).
# Only the "Client" rank continues past this line.
initialize()

if __name__ == "__main__":
    try:
        # Connect the Client rank to the Scheduler (Rank 0)
        with Client() as client:
            print(f"Dashboard available at: {client.dashboard_link}")

        # 2. The Science Logic (Exactly the same!)
        print("Building the task graph...")
        data = da.random.random((500, 2000, 2000), chunks=(100, 1000, 1000))

        # 3. Execution (Exactly the same!)
        # But now, this math is distributed across the entire MPI allocation
        print("Computing the mean...")
        result = data.mean().compute()

        print(f"The mean is: {r
```

```
(cee690) nc153 $ mpirun -n 4 python dask_distributed.py
2026-02-23 22:29:41,130 - distributed.http.proxy - INFO
web server please install jupyter-server-proxy: python -
Dashboard available at: http://10.183.16.94:8787/status
Building the task graph...
Computing the mean...
The mean is: 0.5000080418936698
```

Dask + Xarray

```
# 3. Create the chunked Dask data
print("\nGenerating distributed data...")
raw_dask_array = da.random.random((1000, 180, 360), chunks=(100, 45, 90))

# 4. Wrap it in Xarray
climate_data = xr.DataArray(
    raw_dask_array,
    dims=['time', 'lat', 'lon'],
    coords={
        'time': np.arange(1000),
        'lat': np.linspace(-90, 90, 180),
        'lon': np.linspace(-180, 180, 360)
    },
    name='surface_temperature'
)

# 5. The Science Logic
# We want the global average over time
print("Building the task graph for global time-series...")
time_series = climate_data.mean(dim=['lat', 'lon'])

# 6. Execution across the MPI Workers
print("Executing across MPI cluster...")
final_result = time_series.compute()
```

```
(cee690) nc153 $ mpirun -n 4 python dask_xarray_distributed.py
2026-02-23 22:40:38,058 - distributed.http.proxy - INFO - To ro
web server please install jupyter-server-proxy: python -m pip i
Dashboard available at: http:8787/status
```

```
Generating distributed data...
Building the task graph for global time-series...
Executing across MPI cluster...
```

```
--- First 5 days of Global Mean Temp ---
[0.50089585 0.50172375 0.50017918 0.49907523 0.50081044]
```

**Using low-level distributed
parallelism for day-to-day
spatial data analysis**

Back to the NED/3DEP 1/3 arcsec

```
USGS_NED_13_n40w105_IMG.img
USGS_NED_13_n40w105_IMG.img.aux.xml
USGS_NED_13_n40w105_IMG_meta.html
USGS_NED_13_n40w105_IMG_meta.txt
USGS_NED_13_n40w105_IMG_meta.xml
USGS_NED_13_n40w105_IMG.zip
USGS_NED_13_n40w106_IMG.img
USGS_NED_13_n40w106_IMG.img.aux.xml
USGS_NED_13_n40w106_IMG_meta.html
USGS_NED_13_n40w106_IMG_meta.txt
USGS_NED_13_n40w106_IMG_meta.xml
USGS_NED_13_n40w106_IMG.zip
USGS_NED_13_n40w107_IMG.img
USGS_NED_13_n40w107_IMG.img.aux.xml
USGS_NED_13_n40w107_IMG_meta.html
USGS_NED_13_n40w107_IMG_meta.txt
USGS_NED_13_n40w107_IMG_meta.xml
USGS_NED_13_n40w107_IMG.zip
USGS_NED_13_n40w108_IMG.img
USGS_NED_13_n40w108_IMG.img.aux.xml
USGS_NED_13_n40w108_IMG_meta.html
USGS_NED_13_n40w108_IMG_meta.txt
USGS_NED_13_n40w108_IMG_meta.xml
(cee690) nc153 $
```

```
USGS_NED_13_n71w159_IMG_meta.xml
USGS_NED_13_n71w159_IMG.zip
USGS_NED_13_n72w155_IMG.img
USGS_NED_13_n72w155_IMG_meta.html
USGS_NED_13_n72w155_IMG_meta.txt
USGS_NED_13_n72w155_IMG_meta.xml
USGS_NED_13_n72w155_IMG.zip
USGS_NED_13_n72w156_IMG.img
USGS_NED_13_n72w156_IMG_meta.html
USGS_NED_13_n72w156_IMG_meta.txt
USGS_NED_13_n72w156_IMG_meta.xml
USGS_NED_13_n72w156_IMG.zip
USGS_NED_13_n72w157_IMG.img
USGS_NED_13_n72w157_IMG_meta.html
USGS_NED_13_n72w157_IMG_meta.txt
USGS_NED_13_n72w157_IMG_meta.xml
USGS_NED_13_n72w157_IMG.zip
USGS_NED_13_n72w158_IMG.img
USGS_NED_13_n72w158_IMG_meta.html
USGS_NED_13_n72w158_IMG_meta.txt
USGS_NED_13_n72w158_IMG_meta.xml
USGS_NED_13_n72w158_IMG.zip
USGS_NED_13.vrt
```

```
(cee690) nc153 $ du -sh USGS_NED_13/
879G      USGS_NED_13/
```

1011 files

V0 - Split up the reading/ postprocessing

```
#Initialize a file to write the localized computed means to
wdir = f'/cwork/{netid}/workspace'
os.system(f'mkdir -p {wdir}')
file = f'{wdir}/rank_{rank}'
fp = open(file, 'w')

#Initialize time variables
count_files = 0
sum_time

import numpy as np
import glob

#Read list of files
netid = 'nc153'
wdir = f'/cwork/{netid}/workspace'
files = glob.glob(f'{wdir}/*')
for file in files:
    tic = time.time()
    zsum = 0.0
    #Extract npixels
    npixels = 0
    geolocalization = 0
    for file in files:
        fp = open(file, 'r')
        #Read data
        for line in fp:
            #Compute npixels
            npixels += int(line[:-1].split(',')[1])
            #Compute zsum
            zsum += float(line[:-1].split(',')[2])
        elevation = zsum/npixels
    #Compute elevation mean over Contiguous United States
    print(f"Average elevation over CONUS is {elevation} meters")
```

V1 - More MPI flavor to it

```
for file in files[rank::size]:
    tic = time.time()
    #Extract cell geographic location
    geoloc = file.split("_")[5]
    #Read in the file
    data = rasterio.open(file).read(1)
    #Compute the mask
    mask = data > -9999.0
    npixels += np.sum(mask)
    zsum += np.sum(data[mask])
    #Print current cell location and time it took
    count_files += 1
    sum_time += time.time()-tic
    average_time = sum_time/count_files
    print(f'{rank},{geoloc},{average_time}', flush=True)

#Gather the data
zsum_final = comm.reduce(zsum, op=MPI.SUM, root=0)
npixels_final = comm.reduce(npixels, op=MPI.SUM, root=0)
if rank == 0:
    elevation = zsum_final/npixels_final
    print(f"Average elevation over CONUS is {elevation} meters")
```

Average elevation over CONUS is 587.2528633372601 meters

Does it matter?

1. 1 core/thread - 50 minutes

2. 32 cores/threads - 4.2 minutes

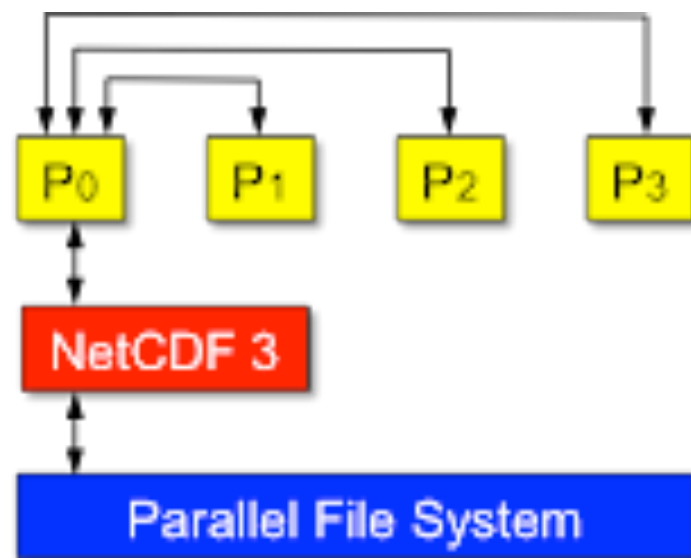
- Why is it only ~12 times faster and not 32?
- The ability to scale your workflow using MPI is key. It accelerates processing and analysis and it accelerates debugging.
- Learning and leveraging this in ~2013 is likely strongly connected to my own career/academic success.

I/O on parallel systems

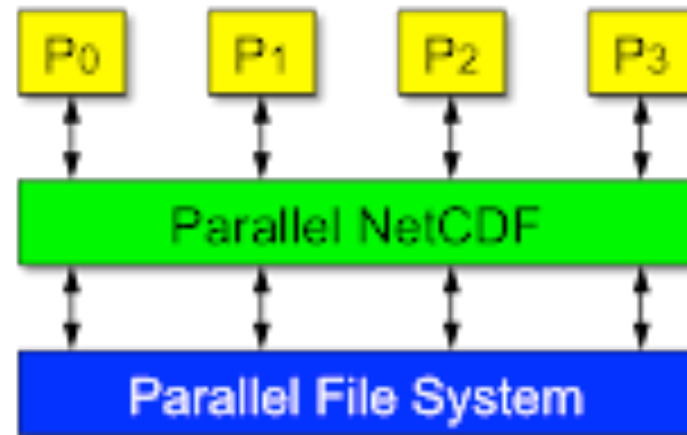
MPI I/O

- Sub-module of the MPI standard
- Allows multiple processes to open the same file and define “views” of it.
- Each rank determines its own offset in storage and writes only to that.

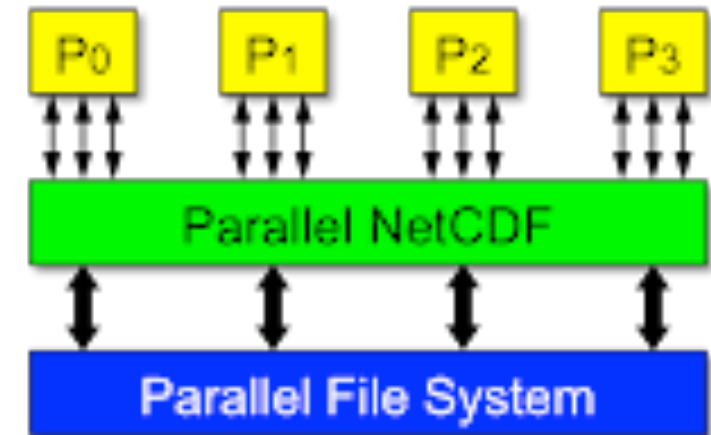
Parallel NetCDF



(a)



(b)

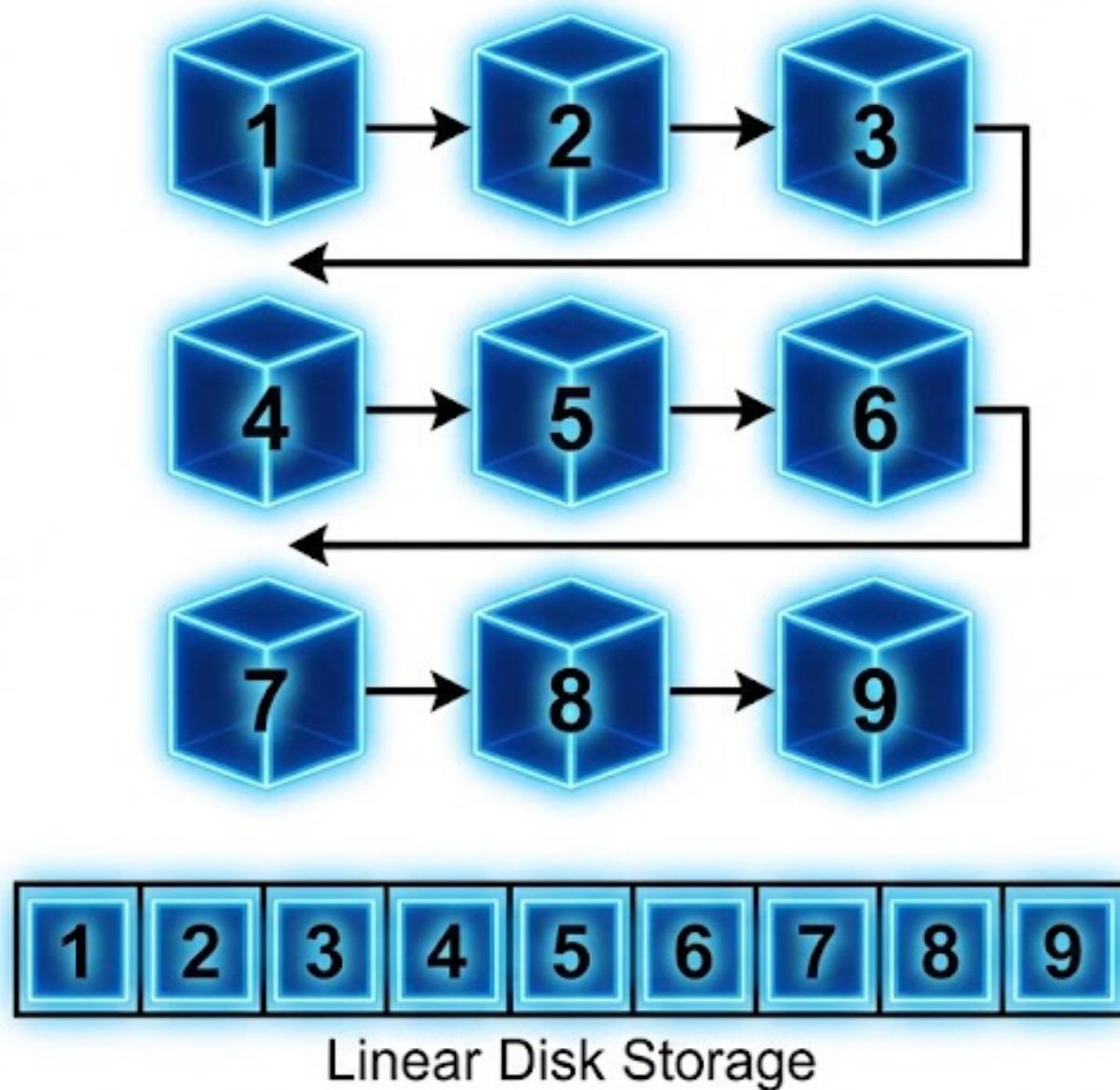


(c)

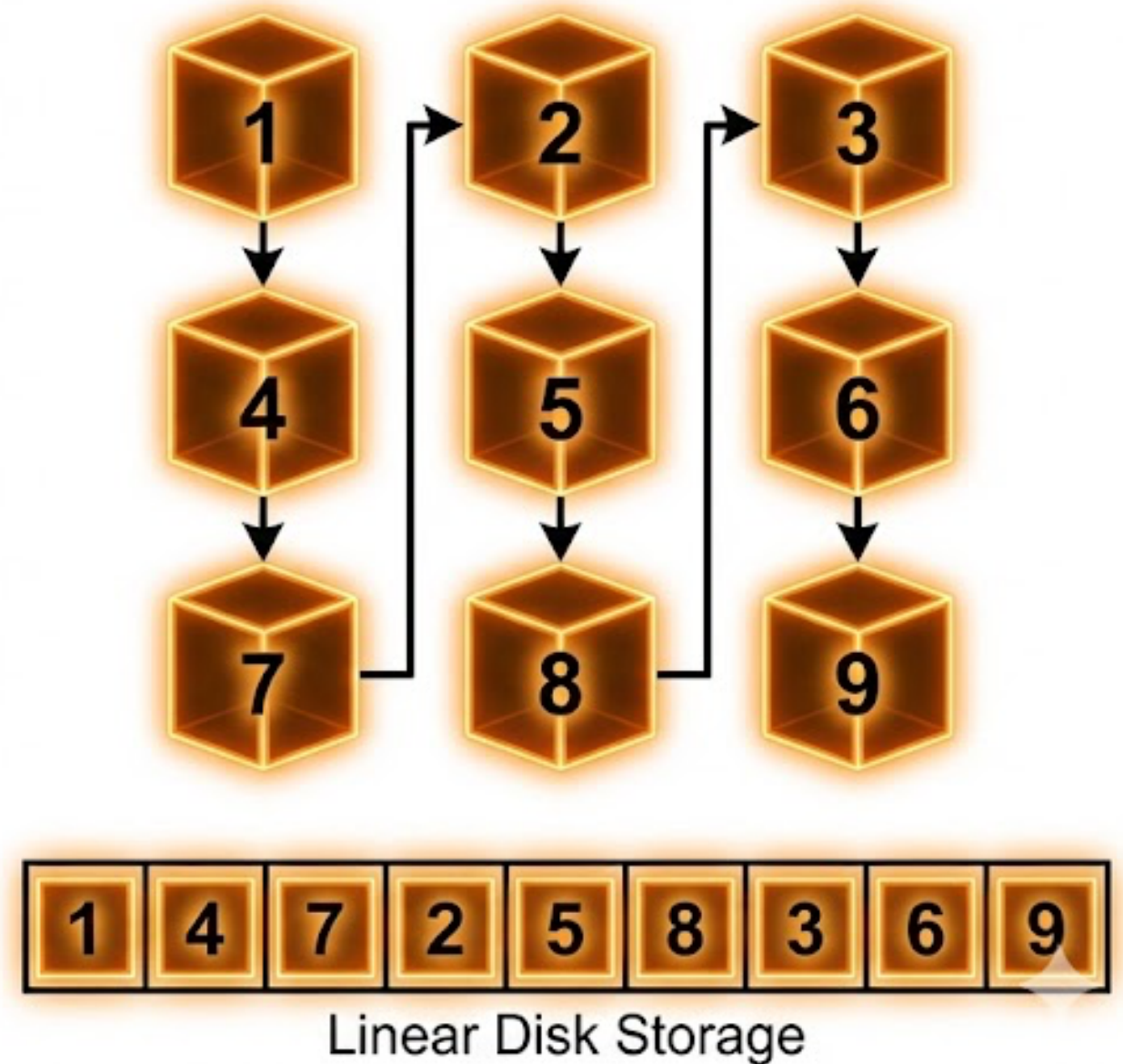
How are arrays written on disk?

Data Storage Layouts: 2D Grid to 1D Linear Memory

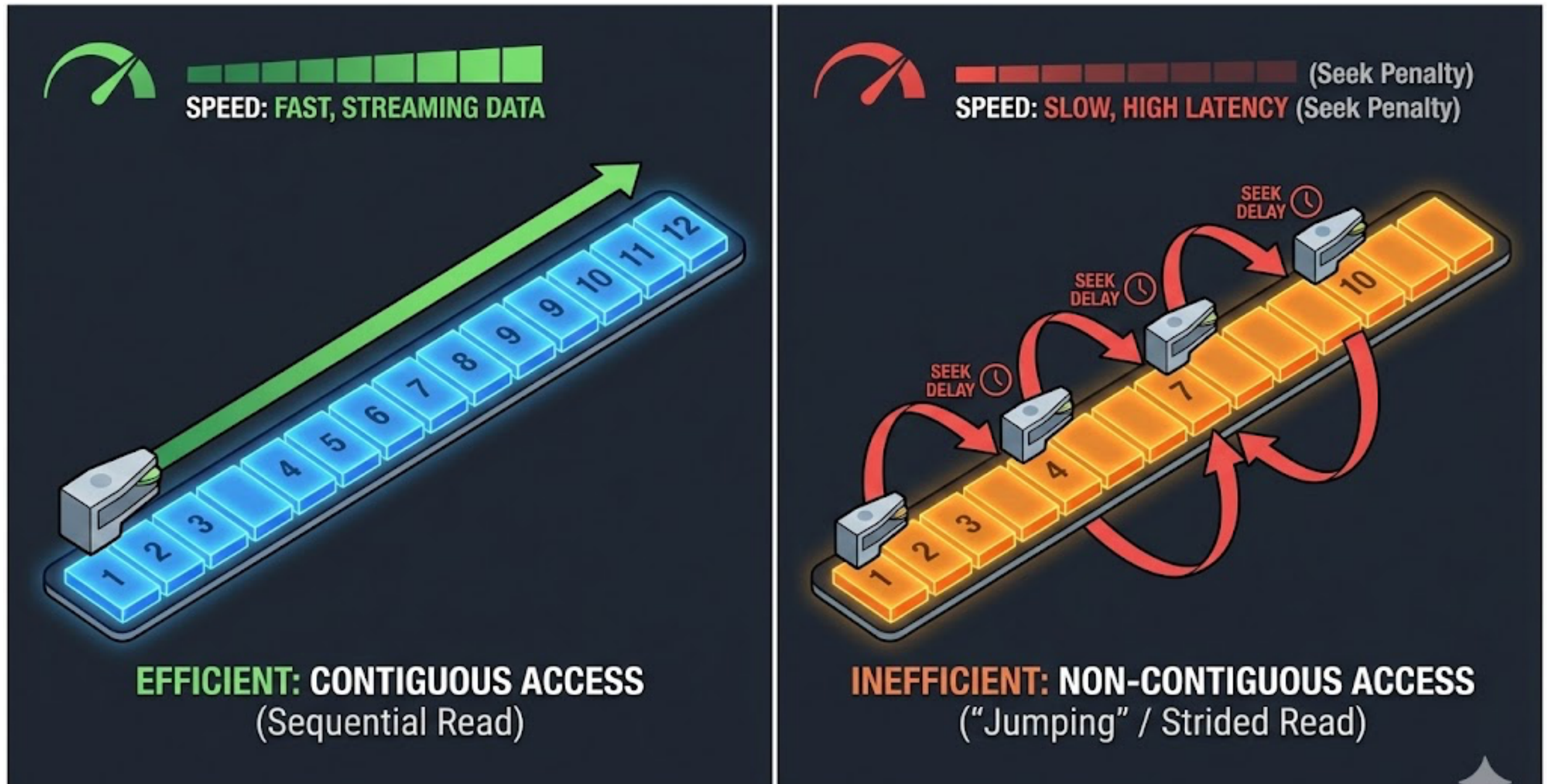
Row-Major Order (C, Python, NetCDF)



Column-Major Order (Fortran, MATLAB)

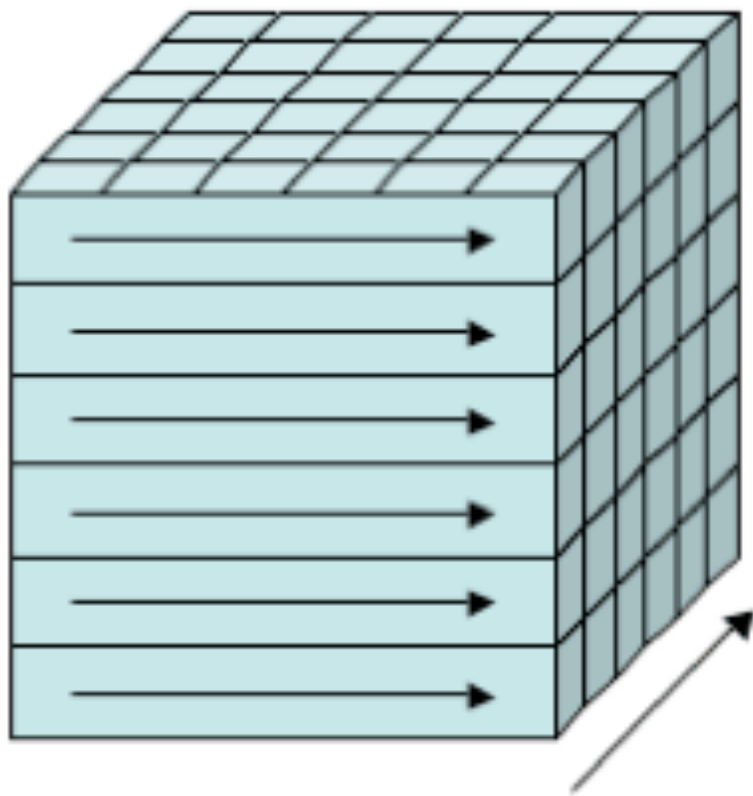


What happens if we query non-contiguous memory?

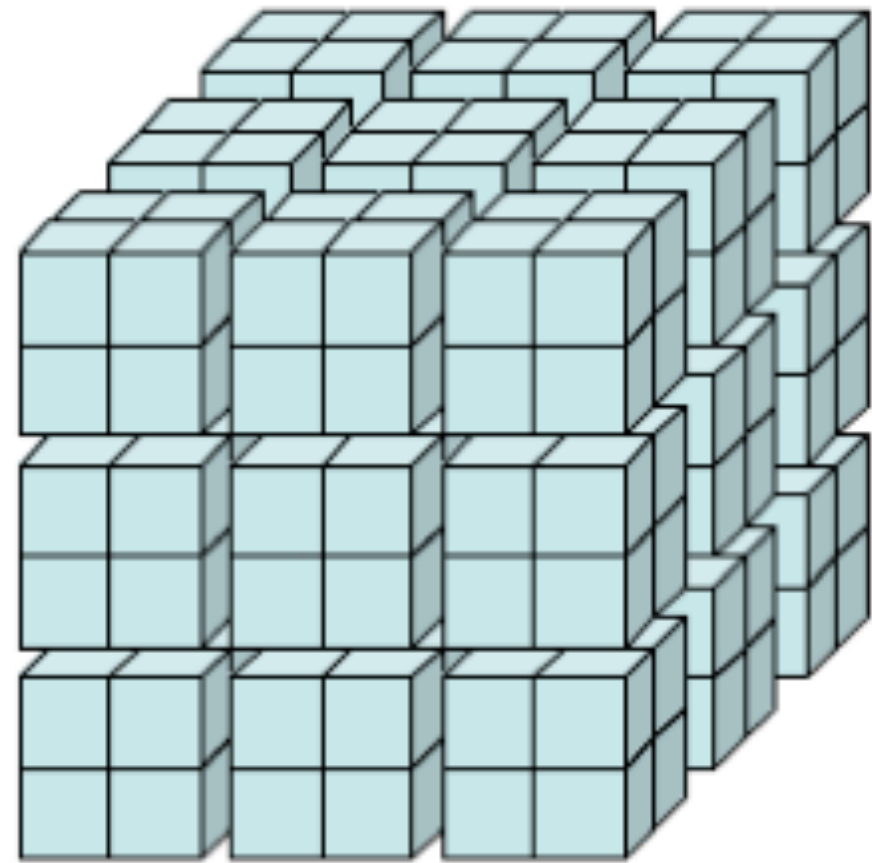


Disk heads or memory controllers waste significant time 'seeking' when accessing data that is not physically adjacent.

Chunking in netcdf4



index order



chunked

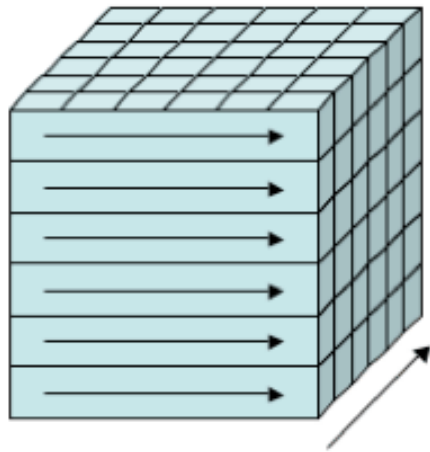
Example data

```
(cee690) nc153 $ ncdump -h era_unchunked.nc
netcdf era_unchunked {
dimensions:
    time = 444 ;
    latitude = 241 ;
    longitude = 480 ;
variables:
    float t2m(time, latitude, longitude) ;
        t2m:_FillValue = -1.e+34f ;
        t2m:long_name = "temperature at 2m" ;
        t2m:units = "degC" ;
        t2m:history = "From /dev/shm/monthly_an_sfc.nc" ;
        t2m:missing_value = -1.e+34f ;
    double time(time) ;
        time:_FillValue = NaN ;
        time:long_name = "time" ;
        time:time_origin = "1-JAN-1979" ;
        time:axis = "T" ;
        time:standard_name = "time" ;
        time:bounds = "time_bnds" ;
        time:units = "days since 1979-01-01" ;
        time:calendar = "GREGORIAN" ;
    float latitude(latitude) ;
        latitude:_FillValue = NaNf ;
        latitude:units = "degrees_north" ;
        latitude:long_name = "latitude" ;
        latitude:point_spacing = "even" ;
        latitude:axis = "Y" ;
        latitude:standard_name = "latitude" ;
    float longitude(longitude) ;
        longitude:_FillValue = NaNf ;
        longitude:units = "degrees_east" ;
        longitude:long_name = "longitude" ;
        longitude:modulo = " " ;
        longitude:point_spacing = "even" ;
        longitude:axis = "X" ;
        longitude:standard_name = "longitude" ;

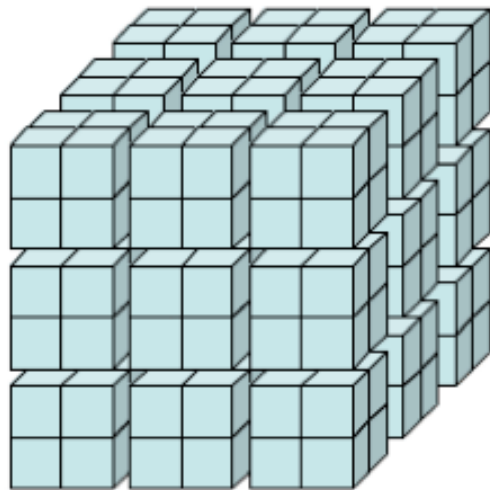
// global attributes:
    :history = "FERRET V7.01 (beta/optim) 29-Sep-16" ;
    :Conventions = "CF-1.6" ;
    :title = "ERA-Interim" ;
}
```

Example 1: Unchunked

```
nc153 $ nccopy -k netCD4 -u -c / era_interim_monthly_197901_201512_t2m.nc era_unchunked.nc
```



index order



chunked

```
import netCDF4 as nc
import time
file = 'era_unchunked.nc'
fp = nc.Dataset(file)

#Query time dimension
tic = time.time()
data = fp['t2m'][:,0,0]
print('Time dimension',time.time() - tic)

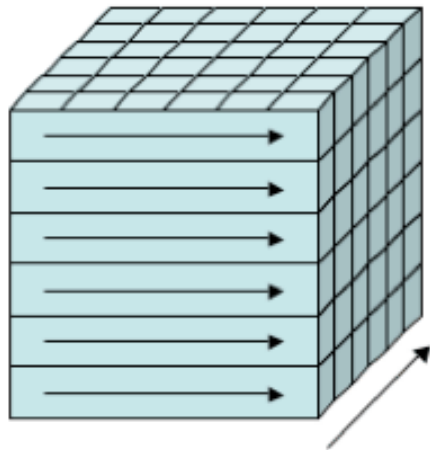
#Query space dimensions
tic = time.time()
data = fp['t2m'][0,:,:]
print('Space dimensions',time.time() - tic)
~
```

```
(cee690) nc153 $ python query_unchunked.py
Time dimension 2.857297658920288
Space dimensions 0.002043485641479492
```

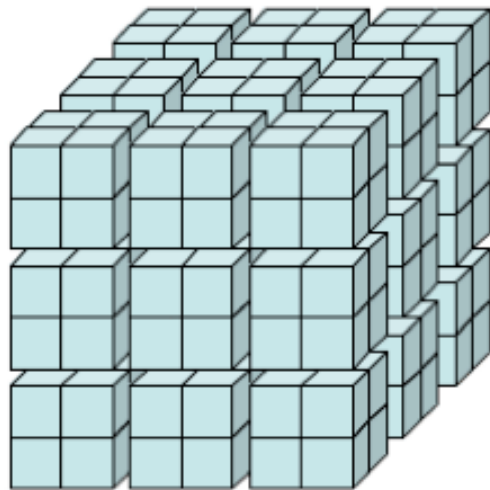
Note: You can also use tools like xarray and cdo to rechunk

Example 2: “Time” chunks

```
$ nccopy -k netCDF4 -c time/256,latitude/5,longitude/5 era_unchunked.nc era_time_chunks.nc
```



index order



chunked

```
import netCDF4 as nc
import time
file = 'era_time_chunks.nc'
fp = nc.Dataset(file)

#Query time dimension
tic = time.time()
data = fp['t2m'][:,0,0]
print('Time dimension',time.time() - tic)

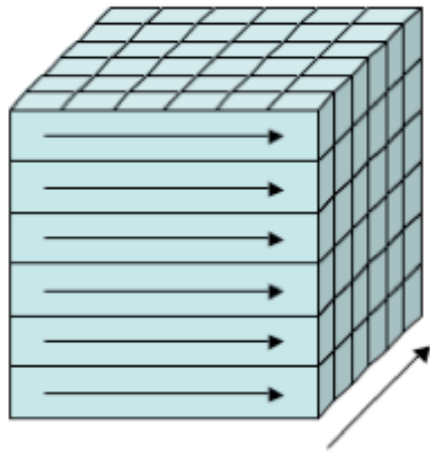
#Query space dimensions
tic = time.time()
data = fp['t2m'][0,:,:]
print('Space dimensions',time.time() - tic)

fp.close()
```

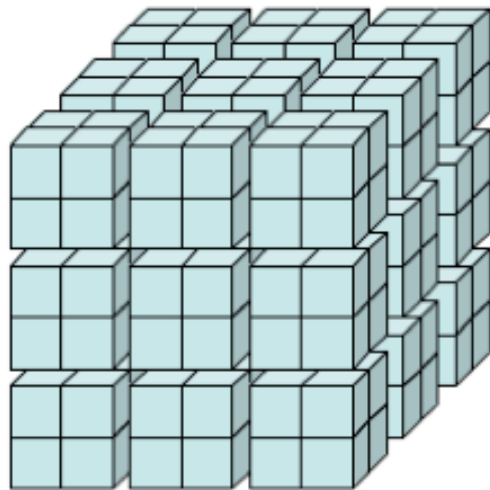
```
[(cee690) nc153 $ python query_time.py
Time dimension 0.00563812255859375
Space dimensions 2.669614791870117
```

Example 3: “Space” chunks

```
$ nccopy -k netCDF4 -c time/256,latitude/5,longitude/5 era_unchunked.nc era_time_chunks.nc
```



index order



chunked

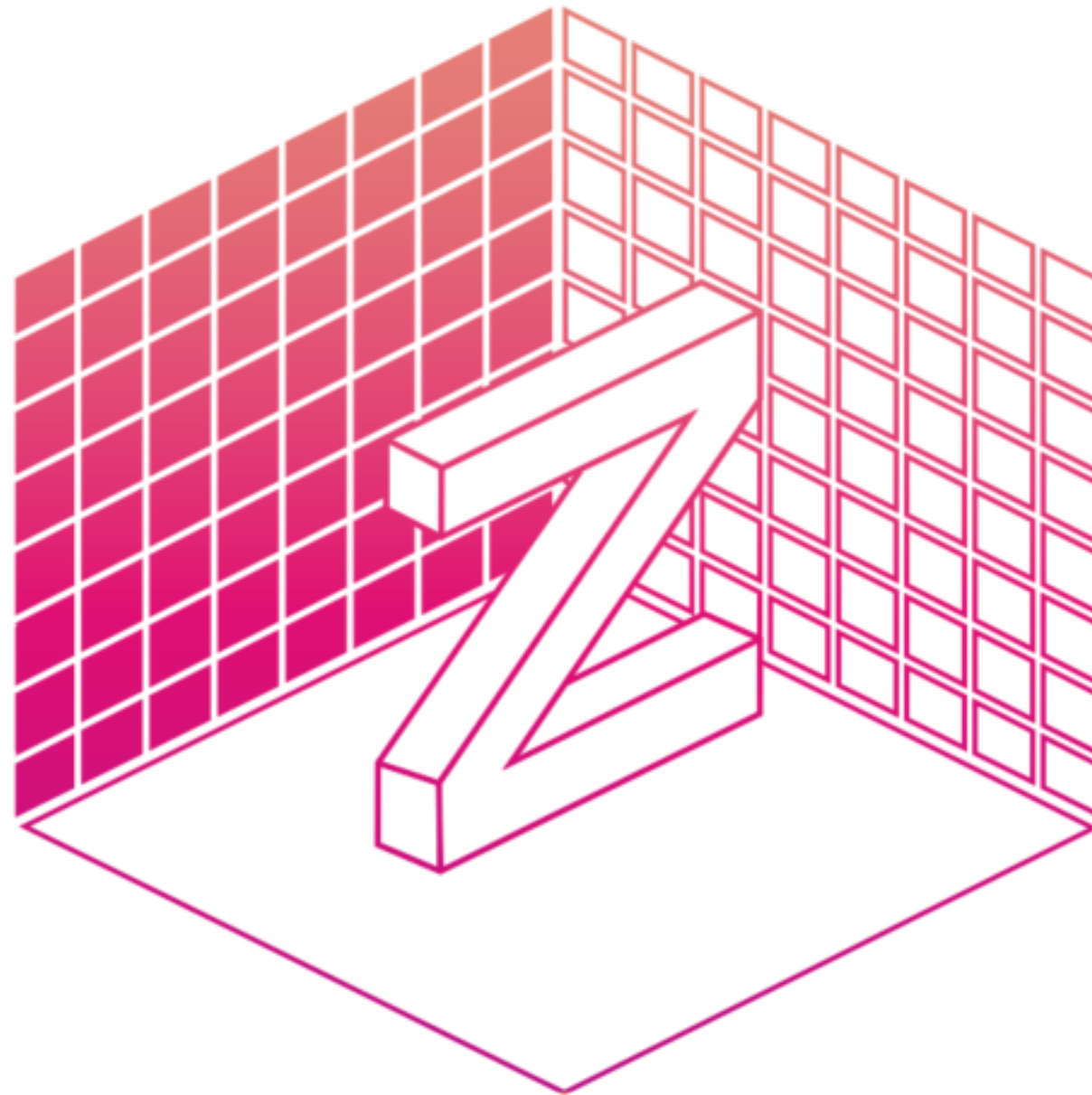
```
import netCDF4 as nc
import time
file = 'era_unchunked.nc'
fp = nc.Dataset(file)

#Query time dimension
tic = time.time()
data = fp['t2m'][:,0,0]
print('Time dimension',time.time() - tic)

#Query space dimensions
tic = time.time()
data = fp['t2m'][0,:,:]
print('Space dimensions',time.time() - tic)
~
```

```
(cee690) nc153 $ python query_space.py
Time dimension 3.4145185947418213
Space dimensions 0.022449731826782227
```

Zarr



Why Zarr?

- Native Parallelism (No "File Locking"))
- Cloud-Native Design
- Consolidated Metadata
- Flexible Storage Backends
- Optimized for Dask/Xarray

What does Zarr look like?

```
[(cee690) nc153 $ cd era_interim.zarr/  
[(cee690) nc153 $ ls  
latitude longitude t2m time
```

```
[(cee690) nc153 $ tree
```

```
.  
├── latitude  
│   └── 0  
├── longitude  
│   └── 0  
└── t2m  
    ├── 0.0.0  
    ├── 0.0.1  
    ├── 0.0.2  
    ├── 0.0.3  
    ├── 0.1.0  
    ├── 0.1.1  
    ├── 0.1.2  
    ├── 0.1.3  
    ├── 0.2.0  
    ├── 0.2.1  
    ├── 0.2.2  
    ├── 0.2.3  
    ├── 0.3.0  
    ├── 0.3.1  
    └── 0.3.2
```

Zarr is basically chunking at the directory level instead of in the file.